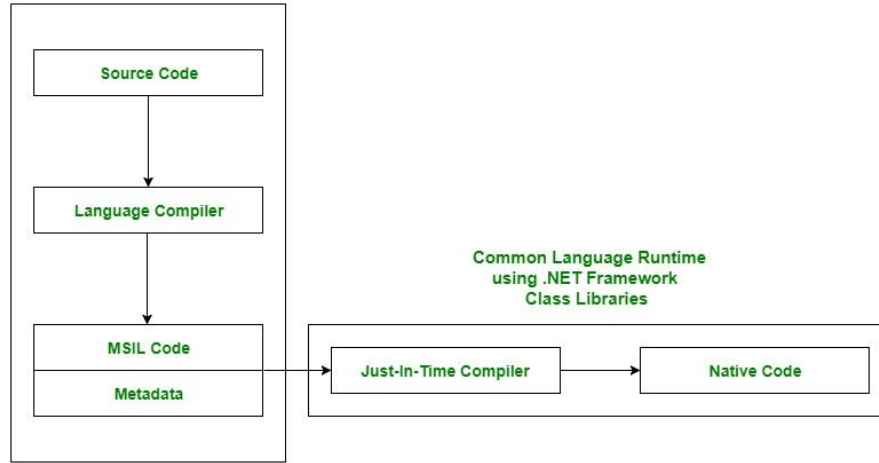




C# Execution

C# Program Execution



- Suppose you have written a C# program and save it in a file which is known as the Source Code.
- Language specific compiler compiles the source code into the **MSIL (Microsoft Intermediate Language)** which is also known as the **CIL (Common Intermediate Language)** or **IL (Intermediate Language)** along with its metadata. *Metadata* includes all the types, actual implementation of each function of the program. MSIL is machine-independent code.
- Now CLR comes into existence. CLR provides the services and runtime environment to the MSIL code. Internally CLR includes the JIT (Just-In-Time) compiler which converts the MSIL code to machine code which further executed by CPU. CLR also uses the .NET Framework class libraries. Metadata provides information about the programming language, environment, version, and class libraries to the CLR by which CLR handles the MSIL code. As CLR is common so it allows an instance of a class that written in a different language to call a method of the class which written in another language.

Component of CLR

Common Language Specification (CLS):

- It is responsible for converting the different .NET programming language syntactical rules and regulations into CLR understandable format. Basically, it provides Language Interoperability. Language Interoperability means providing execution support to other programming languages also in .NET framework.

Language Interoperability can be achieved in two ways :

Managed Code: The MSIL code which is managed by the CLR is known as the Managed Code. For managed code CLR provides three .NET facilities:

Unmanaged Code: Before .NET development, programming languages like.COM Components & Win32 API do not generate the MSIL code. So these are not managed by CLR rather managed by Operating System.

Component of CLR

Common Type System (CTS):

- Every programming language has its own data type system, so CTS is responsible for understanding all the data type systems of .NET programming languages and converting them into CLR understandable format which will be a common format.
- There are 2 Types of CTS that every .NET programming language have :
 - a. **Value Types:** Value Types will store the value directly into the memory location. These types work with stack mechanisms only. CLR allows memory for these at Compile Time.
 - b. **Reference Types:** Reference Types will contain a memory address of value because the reference types won't store the variable value directly in memory. These types work with Heap mechanism. CLR allot memory for these at Runtime.

Garbage Collector:

- It is used to provide the Automatic Memory Management feature. If there was no garbage collector, programmers would have to write the memory management codes which will be a kind of overhead on programmers.

JIT(Just In Time Compiler):

- It is responsible for converting the CIL(Common Intermediate Language) into machine code or native code using the Common Language Runtime environment.